



Tajweed Exercises

Data format and exercise types — a guide for the app developer

German Method · revision of 23 September 2026

This document describes the exercise corpus that ships as JSON: what an exercise looks like, what the seven interaction types mean, how the question text is produced, and what a client has to do to render Qur'anic Arabic correctly. Everything in here is generated from the files themselves, so the numbers match what you receive.

2036 exercises · **40** lessons · **7** rules · **7** interaction types · **23** question wordings · **2809** audio recordings

Files: `json/*.json`, `questions.json`, `advanced.json` — downloadable as one ZIP from the app preview.

Contents

1. What you are given
2. The exercise object, field by field
3. The four classifier keys
4. Where the question text comes from
5. The seven interaction types
6. Highlighting the answer: the `spots` field
7. Audio
8. Combined lessons for advanced learners
9. Rendering Qur'anic Arabic
10. Invariants worth asserting
11. Appendix: inventory

1 · What you are given

Seven files, one per block of the syllabus. Each file is a JSON array of exercise objects; there is no envelope, no metadata header. Exercise ids are unique across all seven files, so you can merge them into a single table and key on `id`.

File	Lessons	Exercises	Audio files
json/tafkheem.json	1–9	575	791
json/qalqala.json	10–14	319	439
json/idgham.json	15–21	288	503
json/ikhfa-idgham.json	22–26	294	522
json/iqlab.json	27–28	128	202
json/madd.json	29–34	256	176
json/waqf.json	35–40	176	176

Two further files sit next to them:

- `questions.json` — the wording of every question, in German and English. An exercise carries no prose of its own; see section 4.
- `advanced.json` — which exercises make up the condensed lessons for advanced learners; see section 8.

Encoding. All files are UTF-8 without a BOM. Arabic text is stored in the Uthmani orthography of the Madinah muṣḥaf, in NFC, and contains no characters outside the Basic Multilingual Plane. That last point matters for the `spots` offsets: one character equals one code point equals one UTF-16 code unit, so the same integers are correct in JavaScript, Swift, Kotlin and Python alike.

Getting the files

The download button on the app preview at `tajweed-exercises.vercel.app` packs everything into `tajweed-aufgaben.zip`. Single files are also served directly, for example `/json/tafkheem.json` or `/questions.json`. The same files live in the repository under `json/`.

2 · The exercise object, field by field

Field	Type	Present	Meaning
id	integer	always	Unique across the whole corpus. Also the stem of the audio filenames.
rule	enum	always	tafkheem · qalqala · idgham · ikhfa · iqḷab · madd · waqf
lesson	integer	always	1–40, the lesson of the syllabus.
task_type	enum	always	How the learner interacts. Seven values, section 5.
question_type	enum	always	Which question is asked. Seventeen values.
question_id	string	always	Key into questions.json. Finer than question_type, section 4.
multiple	boolean	only select	true means the learner may tick more than one option.
modality	enum	always	audio — the item is meant to be heard; text — read only.
sura	integer or null	always	Sura number when the subject is a Qur’anic verse, else null.
verse	integer or null	always	Verse number, same condition.
subject	object or null	always	The word or verse the question is about. null only for matching.
items	array	only matching	The words to be classified.
options	array	always	The answer choices. Empty array for mark_verse.
answer	varies	always	Shape depends on task_type, section 5.
pattern	string or null	optional	The exact tajweed pattern targeted, colon-separated. Authoring aid, see below.

Inside `subject`, `options[]` and `items[]`

Field	Type	Where	Meaning
text	string	always	The Arabic content, or a fixed token (see next table).
spots	array	optional	Character spans where the rule applies, section 6.
audio	string or null	optional	Filename of the recording, section 7.
id	integer	options and items	Stable within the exercise. The answer points at these.

Fixed tokens in `text`

Most `text` values are Arabic and go on screen unchanged. A handful are fixed ASCII tokens that your client localises itself — they are deliberately not German or English in the data, so that one corpus serves

every language.

Token	Meaning
yes / no	The two options of every yes_no exercise.
start / mid / end	Position in the word, for select_position.
none	“does not occur” — the rule is absent, or none of the words qualifies.
-	“no pattern” — in matching, the bucket for a word that fits none.
digits	"0" ... "6" in select_count; count them as numbers, not as Arabic.

Do not display a token verbatim. `none` needs different wording depending on the exercise: “does not occur” after a counting question, “none of the words” in a word selection, “neither of them” in the two-way pause exercise of lesson 40. Our own front end switches on `task_type` and `question_id` to pick the phrasing.

A note on `pattern`

`pattern` lists the shapes the exercise targets, separated by colons, with  standing in as a dotted circle carrier — for instance  or . It was an authoring aid and it is not always exact: in a few exercises it names a vowel the word does not carry. **Do not derive anything from it.** Use it, if at all, as a caption for the teacher’s view. For “where does the rule apply”, use `spots`, which is checked against the answer key.

TypeScript shape

```
type Rule = 'tafkheem'|'qalqala'|'idgham'|'ikhfa'|'iqlab'|'madd'|'waqf';
type TaskType = 'yes_no'|'select_count'|'select_position'|'select'
               | 'match'|'mark_verse'|'matching';

interface Spot { start: number; end: number; text: string }

interface Piece {
  id?: number;           // options[] and items[] only
  text: string;
  spots?: Spot[];
  audio?: string | null;
}

interface Pair { item: number; option: number }

interface Exercise {
  id: number;
  rule: Rule;
  lesson: number;       // 1..40
  task_type: TaskType;
  question_type: string;
  question_id: string;  // key into questions.json
  multiple?: boolean;   // task_type === 'select'
  modality: 'audio' | 'text';
  sura: number | null;
  verse: number | null;
  subject: Piece | null; // null only when task_type === 'matching'
  items?: Piece[];       // task_type === 'matching'
  options: Piece[];       // [] when task_type === 'mark_verse'
  answer: number[] | string[] | Pair[];
  pattern?: string | null;
}
```

A complete exercise

```
{
  "id": 1,
  "rule": "tafkheem",
  "lesson": 1,
  "task_type": "yes_no",
  "question_type": "has_rule",
  "question_id": "has_rule.word",
  "modality": "audio",
  "sura": null,
  "verse": null,
  "subject": {
    "text": "قَسَمَ",
    "spots": [
      {
        "start": 0,
        "end": 2,
        "text": "قَ"
      }
    ]
  },
  "audio": "1.wav"
},
"options": [
  {
    "id": 1,
    "text": "yes"
  },
  {
    "id": 2,
    "text": "no"
  }
],
"answer": [
  1
],
"pattern": "قَ"
}
```

3 · The four classifier keys

An exercise is described by four enums. Together they determine both what is shown and what the learner does; nothing else in the record is prose.

rule — which tajweed rule

Value	Name we use	Lessons	What it is
tafkheem	Tafkheem	1–9	Heavy consonants and the rules of the Rā' and the name Allah.
qalqala	Qalqalah	10–14	The echoing release of ق ط ب ج د when they carry no vowel.
idgham	Idgham	15–21, 24–25	One letter merging into the next.
ikhfa	Ikhfa	22–23, 26	A nasal letter hidden before certain consonants.
iqlab	Iqlab	27–28	Nūn or tanwīn turning into a Mīm before Bā'.
madd	Madd	29–34	Prolongation beyond the plain long vowel.
waqf	Waqf	35–40	The pause marks of the muṣḥaf.

Spelling. We transliterate the same way in German and in English, oriented on English conventions: long ī as ee, ح as kh, no diacritics. Hence Tafkheem, Qalqalah, Ikhfa, Noon, Meem, Tanween, Saktah, Muanaqah. If your UI shows rule names, take them from this table rather than inventing your own.

lesson — position in the syllabus

1 to 40. The syllabus is strictly cumulative: an exercise in lesson n never requires a rule first taught after n . You can rely on that when ordering content or gating progress.

task_type — the interaction

Seven values. They decide the shape of `answer` and the widget you build. Section 5 covers each one.

question_type — the question asked

Seventeen values. Several interaction types share a question type and vice versa; the pairing is fixed and listed in the next section.

4 · Where the question text comes from

No exercise contains a question. The sentence is produced from `question_id` against `questions.json`, so that one corpus serves both languages and so that a teacher can rewrite the wording without anyone touching the data.


```

{
  "placeholders": {
    "rule": "Name der Regel (Tafkheem, Qalqalah, Idgham, Ikhfa, Iqlab, Madd, Waqf)",
    "sign": "das Halt-Zeichen der Aufgabe, z. B. ◌̣",
  },
  "rules": {
    "tafkheem": "Tafkheem",
    "...": "..."
  },
  "questions": [
    {
      "id": "has_rule.word",
      "de": "Enthält dieses Wort ein {rule}?",
      "en": "Does this word contain {rule}?",
      "question_type": "has_rule",
      "task_type": [
        "yes_no"
      ],
      "note": "Ja/Nein zu einem einzelnen Wort.",
      "rules": [
        "tafkheem",
        "qalqala",
        "idgham",
        "ikhfa",
        "iqlab"
      ],
      "lessons": [
        1,
        27
      ],
      "count": 116,
      "example": 185
    }
  ]
}

```

Two placeholders appear in the text: `{rule}` becomes the display name of the exercise's rule, `{sign}` becomes its pause mark, taken from `pattern`. Substitute after escaping, so that teacher-authored text cannot inject markup.

```

function questionText(ex, catalogue, lang /* 'de' | 'en' */) {
  const entry = catalogue[ex.question_id];
  if (!entry) return ''; // unknown id: show nothing
  return escapeHtml(entry[lang] ?? entry.de)
    .split('{rule}').join(escapeHtml(RULE_NAME[ex.rule]))
    .split('{sign}').join(signMarkup(ex.pattern));
}

```

Why question_id and not question_type ? Because the same question type is worded differently depending on context. `has_rule` asks “does this word contain Idgham?” for a single word but “do these words contain Idgham?” for a word pair, and for Madd it does not name the rule at all — it asks whether the word contains a prolongation longer than two counts, because in this course the plain long vowel is not itself a Madd. Four wordings, one question type. Keying on `question_id` saves you from re-deriving that.

The wordings

question_id	task_type	German wording	n	Lessons
has_rule.word	yes_no	Enthält dieses Wort ein {rule}?	116	1–27
has_rule.phrase	yes_no	Enthalten diese Wörter ein {rule}?	195	9–28
has_rule.madd.word	yes_no	Enthält dieses Wort eine Dehnung, die länger als zwei Einheiten ist?	104	29–33
has_rule.madd.phrase	yes_no	Enthalten diese Wörter eine Dehnung, die länger als zwei Einheiten ist?	56	31–32
count_rule	select_count	Wie oft kommt {rule} vor?	457	1–28
count_rule.madd	select_count	Wie oft kommt eine Dehnung vor, die länger als zwei Einheiten ist?	96	29–34
vowel_on_letter	select	Welcher Vokal steht auf dem {rule}-Buchstaben?	61	1–9
vowels_on_letter	select	Welche Vokale stehen auf den {rule}-Buchstaben?	155	1–9
vowel_before_letter	select	Welcher Vokal steht vor dem {rule}-Buchstaben?	70	10–28
vowels_before_letter	select	Welche Vokale stehen vor den {rule}-Buchstaben?	78	10–14
which_letter	select	Welcher Buchstabe löst das {rule} aus?	9	22–23
position_in_word	select_position	An welcher Stelle im Wort steht das {rule}?	112	1–14
match_rule.prompt	match	Welche der folgenden Wörter bilden mit diesem ein {rule}?	150	15–28
match_rule.list	match	Markiere alle Wörter mit {rule}. (Mehrfachauswahl möglich)	11	22–27
match_pattern	matching	Ordne jedes Wort seinem {rule}-Muster zu.	116	1–27
mark_rule_in_verse	mark_verse	Markiere jede Stelle im Vers, an der {rule} vorkommt.	74	24–28
identify_waqf_sign	select	Welches Waqf-Zeichen steht in diesem Vers?	64	35–39
identify_waqf_sign.muanaqah	select	Muanaqah {sign} — an welchem der beiden Wörter darf angehalten werden? (nur an einer der beiden Stellen, nicht an beiden)	16	40
mark_waqf_optional	mark_verse	Markiere jedes Wort vor einem freiwilligen Halt {sign}.	16	35
mark_waqf_better_continue	mark_verse	Markiere jedes Wort, an dem besser weitergelesen wird {sign}.	16	36
mark_waqf_better_pause	mark_verse	Markiere jedes Wort, an dem besser angehalten wird {sign}.	16	37
mark_waqf_forbidden	mark_verse	Markiere jedes Wort, an dem der Halt verboten ist {sign}.	32	38
mark_waqf_mandatory	mark_verse	An welchen Stellen darf man nicht ohne Anhalten weiterlesen?	16	39

5 · The seven interaction types

task_type	n	options	answer	What the learner does
yes_no	471	two fixed options	[id]	Does the rule occur here?
select_count	553	number options	[id]	How many times does it occur?
select_position	112	start / mid / end / none	[id]	Where in the word?
select	453	2–5 options	[id, ...]	Which vowel, letter or pause sign?
match	161	4–5 word options	[id, ...]	Which words form the rule with the reference word?
matching	116	pattern options + items	[{item, option}, ...]	Sort four words into pattern buckets.
mark_verse	170	none	["...", ...]	Mark the spots inside the verse.

Everything below is real data, copied out of the files unchanged.

The simplest form: one word or a two-word pair is shown, the learner decides whether the rule is present. **options** is always exactly the two tokens **yes** and **no**, in that order, with ids 1 and 2 — but read the ids from the data rather than assuming.

answer — an array with exactly one option id.

```
{
  "id": 1,
  "rule": "tafkheem",
  "lesson": 1,
  "task_type": "yes_no",
  "question_type": "has_rule",
  "question_id": "has_rule.word",
  "modality": "audio",
  "sura": null,
  "verse": null,
  "subject": {
    "text": "قَسَمَ",
    "spots": [
      {
        "start": 0,
        "end": 2,
        "text": "قَ"
      }
    ]
  },
  "audio": "1.wav"
},
"options": [
  {
    "id": 1,
    "text": "yes"
  },
  {
    "id": 2,
    "text": "no"
  }
],
"answer": [
  1
],
"pattern": "قَ"
}
```

The subject is a word pair in 251 of these; there the rule only arises across the word boundary, and the wording switches to the plural accordingly (**has_rule.phrase**).

Counting `select_count` 553 exercises

A whole verse is shown and the learner counts the occurrences. The options are consecutive numbers as strings; the window varies per exercise (`0...3` , `1...4` , `3...6`), so render them from the data, never from a fixed range.

answer — an array with exactly one option id — the option whose `text` is the correct number.

```

{
  "id": 5,
  "rule": "tafkheem",
  "lesson": 1,
  "task_type": "select_count",
  "question_type": "count_rule",
  "question_id": "count_rule",
  "modality": "audio",
  "sura": 50,
  "verse": 17,
  "subject": {
    "text": "إِذْ يَتَلَقَّى الْمُتَلَقِّيَانِ عَنِ الْيَمِينِ وَعَنِ الشِّمَالِ قَعِيدٌ ۚ",
    "spots": [
      {
        "start": 11,
        "end": 14,
        "text": "قَعِيدٌ"
      },
      {
        "start": 25,
        "end": 28,
        "text": "قَعِيدٌ"
      },
      {
        "start": 68,
        "end": 70,
        "text": "قَعِيدٌ"
      }
    ],
    "audio": "5.wav"
  },
  "options": [
    {
      "id": 1,
      "text": "1"
    },
    {
      "id": 2,
      "text": "2"
    },
    {
      "id": 3,
      "text": "3"
    },
    {
      "id": 4,
      "text": "4"
    }
  ],
  "answer": [
    3
  ],
  "pattern": "قَعِيدٌ:قَعِيدٌ:قَعِيدٌ"
}

```

Note that `spots` lists exactly as many spans as the correct number. That is not a coincidence: the spans were derived from the rule and then checked against the answer key, and only written when the two agreed. You can use it as a self-check, or to show the learner where the occurrences were after answering.

Position in the word select_position 112 exercises

Four options: `start`, `mid`, `end` and `none`. The fourth is never the correct answer in the current corpus, but it is offered so that the learner cannot infer that the rule must be present somewhere.

answer — an array with exactly one option id.

```
{
  "id": 3,
  "rule": "tafkheem",
  "lesson": 1,
  "task_type": "select_position",
  "question_type": "position_in_word",
  "question_id": "position_in_word",
  "modality": "audio",
  "sura": null,
  "verse": null,
  "subject": {
    "text": "عُنُق",
    "spots": [
      {
        "start": 4,
        "end": 6,
        "text": "قُ"
      }
    ]
  },
  "audio": "3.wav"
},
"options": [
  {
    "id": 1,
    "text": "start"
  },
  {
    "id": 2,
    "text": "mid"
  },
  {
    "id": 3,
    "text": "end"
  },
  {
    "id": 4,
    "text": "none"
  }
],
"answer": [
  3
],
"pattern": "قُ"
}
```

Selection `select` 453 exercises

The broadest type. It covers five question types: which vowel sits on the rule letter, which vowel precedes it, which letter triggers the rule, and which pause sign occurs in the verse. The option texts are therefore sometimes bare diacritics (`َ`, `ُ`, `ِ`, `ْ`), sometimes letters, sometimes whole words.

Always read `multiple`. It is present on every `select` exercise and nowhere else. When it is `false` the answer holds exactly one id; when `true` it may hold one, several, or — via the `none` option — the statement that nothing occurs.

answer — an array of option ids. Exactly one if `multiple` is `false`.


```

{
  "id": 2,
  "rule": "tafkheem",
  "lesson": 1,
  "task_type": "select",
  "question_type": "vowel_on_letter",
  "question_id": "vowel_on_letter",
  "multiple": false,
  "modality": "audio",
  "sura": null,
  "verse": null,
  "subject": {
    "text": "قَبْلَ",
    "spots": [
      {
        "start": 0,
        "end": 2,
        "text": "قُ"
      }
    ]
  },
  "audio": "2.wav"
},
"options": [
  {
    "id": 1,
    "text": ""
  },
  {
    "id": 2,
    "text": "ا"
  },
  {
    "id": 3,
    "text": "ـ"
  },
  {
    "id": 4,
    "text": "ا"
  },
  {
    "id": 5,
    "text": "none"
  }
],
"answer": [
  3
],
"pattern": "قُ"
}

```

A bare diacritic needs a carrier

An option whose text is a single combining mark will not render on its own; it attaches to whatever precedes it. Prefix the dotted circle `U+25CC` when the text consists only of combining characters:

```

const COMBINING = /^[\u0610-\u061A\u064B-\u065F\u0670\u06D6-\u06ED]+$ /;
const display = (t) => COMBINING.test(t) ? '\u25CC' + t : t; // ○~

```

Word matching match 161 exercises

A reference word is shown with four candidate words; the learner picks those that form the rule *together with* the reference word. This is the type where the rule lives at the seam between two words, so the answer is only meaningful as a pair.

Two shapes exist. Usually `subject.text` is a real reference word and the recording is named `<id>_prompt.wav`. In 11 exercises `subject.text` is the placeholder `اختر` (Arabic for “choose”), which means there is no reference word and each candidate stands on its own; `question_id` is then `match_rule.list` rather than `match_rule.prompt`. Check for that placeholder, do not print it.

answer — an array of option ids; it may be a single id, several, or the id of the `none` option.

```

{
  "id": 920,
  "rule": "idgham",
  "lesson": 15,
  "task_type": "match",
  "question_type": "match_rule",
  "question_id": "match_rule.prompt",
  "modality": "audio",
  "sura": null,
  "verse": null,
  "subject": {
    "text": "كَانَ",
    "spots": [
      {
        "start": 5,
        "end": 7,
        "text": "و"
      }
    ]
  },
  "audio": "920_prompt.wav"
},
"options": [
  {
    "id": 1,
    "text": "تَأْتِيكُمْ",
    "spots": [
      {
        "start": 0,
        "end": 3,
        "text": "ع"
      }
    ]
  },
  "audio": "920_1.wav"
},
{
  "id": 2,
  "text": "تَأْتِيهِمْ",
  "spots": [
    {
      "start": 0,
      "end": 3,
      "text": "ع"
    }
  ]
},
  "audio": "920_2.wav"
},
{
  "id": 3,
  "text": "تَجِدَ",
  "spots": [
    {
      "start": 0,
      "end": 3,
      "text": "ع"
    }
  ]
},
  "audio": "920_3.wav"
},
{
  "id": 4,
  "text": "مَنَعُ",
  "audio": "920_4.wav"
},
},

```

```

    {
      "id": 5,
      "text": "none",
      "audio": null
    }
  ],
  "answer": [
    1,
    2,
    3
  ],
  "pattern": "ت ت"
}

```

Reading order is not always prompt first. In exercise 1624 the reference word follows the candidate (أَلَمْ يَأْتِكُمْ) (تَشِيرُ), because that is where the Mīm meets the Bā'. The **spots** on the option tell you which part is marked; if you synthesise the pair yourself for playback or display, take the order from the spots rather than assuming.

Bucket matching matching 116 exercises

Four words are sorted into pattern buckets. This is the only type where `subject` is `null`; the content sits in `items` (the words) and `options` (the buckets). The last bucket is usually the token `-`, “no pattern”, for the word that fits none of them.

answer — an array of `{item, option}` pairs, one per item, covering every item exactly once.

```

{
  "id": 4,
  "rule": "tafkheem",
  "lesson": 1,
  "task_type": "matching",
  "question_type": "match_pattern",
  "question_id": "match_pattern",
  "modality": "audio",
  "sura": null,
  "verse": null,
  "subject": null,
  "items": [
    {
      "id": 1,
      "text": "قَلَّ",
      "spots": [
        {
          "start": 0,
          "end": 2,
          "text": "قَ"
        }
      ],
      "audio": "4_1.wav"
    },
    {
      "id": 2,
      "text": "تَقَمَّ",
      "spots": [
        {
          "start": 2,
          "end": 4,
          "text": "قَ"
        }
      ],
      "audio": "4_2.wav"
    },
    {
      "id": 3,
      "text": "أُقُقْ",
      "spots": [
        {
          "start": 4,
          "end": 6,
          "text": "قَ"
        }
      ],
      "audio": "4_3.wav"
    },
    {
      "id": 4,
      "text": "عَمَلَّ",
      "audio": "4_4.wav"
    }
  ],
  "options": [
    {
      "id": 1,
      "text": "قَ",
      "spots": [
        {
          "start": 0,
          "end": 2,
          "text": "قَ"
        }
      ]
    }
  ]
}

```

```

    }
  ]
},
{
  "id": 2,
  "text": "ق",
  "spots": [
    {
      "start": 0,
      "end": 2,
      "text": "ق"
    }
  ]
},
{
  "id": 3,
  "text": "ق",
  "spots": [
    {
      "start": 0,
      "end": 2,
      "text": "ق"
    }
  ]
},
{
  "id": 4,
  "text": "-"
}
],
"answer": [
  {
    "item": 1,
    "option": 1
  },
  {
    "item": 2,
    "option": 2
  },
  {
    "item": 3,
    "option": 3
  },
  {
    "item": 4,
    "option": 4
  }
]
}

```

Marking in the verse `mark_verse` 170 exercises

A verse is shown and the learner marks the places where the rule applies — there are no discrete options, so `options` is the empty array. This is the type used for the pause-mark lessons 35–39 as well as for Idgham, Ikhfa and Iqlab in the verse.

answer — an array of strings, each an exact substring of `subject.text`.

```
{
  "id": 1500,
  "rule": "idgham",
  "lesson": 24,
  "task_type": "mark_verse",
  "question_type": "mark_rule_in_verse",
  "question_id": "mark_rule_in_verse",
  "modality": "audio",
  "sura": 56,
  "verse": 63,
  "subject": {
    "text": "أَقْرَأْ يُثْمِ مَا تَحْرُثُونَ",
    "spots": [
      {
        "start": 0,
        "end": 18,
        "text": "أَقْرَأْ يُثْمِ مَا"
      }
    ]
  },
  "audio": "1500.wav"
},
"options": [],
"answer": [
  "أَقْرَأْ يُثْمِ مَا"
],
"pattern": "م م"
}
```

The empty array is an answer. In 30 of these exercises the rule does not occur in the verse at all and `answer` is `[]`. That is the correct response — “does not occur” — not missing data. Every other task type has at least one entry in `answer`; only `mark_verse` may be empty.

A span may cover two words, as above: the Idgham runs across the boundary. It may also occur more than once in the verse — the array then lists the same string twice, and you mark the first two occurrences. When a word repeats and only one instance carries the pause mark, prefer the occurrence followed by the sign named in `pattern`. The `spots` field has already resolved all of this; taking the spans from there instead of matching strings yourself is simpler and safer.

6 · Highlighting the answer: the `spots` field

The answer key tells you *what* is right — a number, a vowel, a yes. It does not tell you *where* in the word the rule applies, and for a counting exercise that is exactly what the learner wants to see afterwards. That is what `spots` is for.

It sits next to `text`, in the `subject` and in individual options and items, and it always refers to **its own** `text`:

```
{ "text": "يَضِيقُ",  
  "spots": [ { "start": 0, "end": 3, "text": "يَضِ" },  
              { "start": 5, "end": 7, "text": "قُ" } ] }
```

The rules it obeys:

- `start` and `end` are character offsets, half-open, exactly like `String.prototype.slice`.
- `text.slice(start, end) === spot.text` holds for every span. Assert it once at load; if it ever fails you have an encoding problem, not a data problem.
- Spans are sorted ascending and never overlap; adjacent ones are merged.
- A span covers a letter *with its diacritics* — `يَضِ`, not `ضِ` — and for rules that straddle a word boundary it covers both words.
- No `spots` key means there is nothing to mark. For the 403 exercises whose answer is “does not occur”, that is precisely right.

Rendering is a few lines:

```
function highlight(text, spots) {  
  let out = '', cursor = 0;  
  for (const s of spots ?? []) {  
    out += escapeHtml(text.slice(cursor, s.start))  
      + '<mark>' + escapeHtml(s.text) + '</mark>';  
    cursor = s.end;  
  }  
  return out + escapeHtml(text.slice(cursor));  
}
```

Coverage in the current corpus: 1473 subjects carry spans and 1028 options or items do.

How they were derived. Not from `pattern`, which is unreliable, but from the rule itself — sukūn and verse end for Qalqalah, a nūn-rule analyser for Idgham, Ikhfa and Iqlab, the pause marks for Waqf, the lesson's letter set for Tafkheem, the written form of the prolongation for Madd. Every derivation was then checked against the answer key: the number of spans must equal the counted number, their presence must match a yes or a no, the vowels on them must equal the vowels in the answer. Only what passed that check was written. Where a client and this field disagree, the field is more likely to be right.

7 · Audio

Recordings are named after the exercise id:

Pattern	What it holds
<code><id>.wav</code>	The subject — the single word or the verse.
<code><id>_prompt.wav</code>	The reference word of a match exercise.
<code><id>_1.wav ... <id>_4.wav</code>	The candidate words of match, the items of matching.

The filename is in the record; never construct it yourself. Fixed tokens (`yes` , `none` , `-` , the numerals) never have a recording — there is nothing to speak. 2809 files make up the full set. The 172 exercises with `modality: "text"` have no audio at all and are meant to be read.

Different exercises may share a recording only if they share the id — which they cannot. Identical Arabic text does occur across exercises, and each occurrence has its own file. If you want to save space, deduplicate by text at packaging time and keep a map; do not assume the corpus has done it for you.

8 · Combined lessons for advanced learners

A learner who already knows the material does not need the letters one at a time. `advanced.json` defines condensed lessons that stand in for several ordinary ones. They contain no new exercises: they list ids of existing ones, so the recordings and the answer keys are shared.

```

{
  "lessons": [
    {
      "id": "tafkheem-1-advanced",
      "group": "tafkheem",
      "block": "tafkheem",
      "replaces": [
        1,
        2,
        3,
        4,
        5,
        6,
        7
      ],
      "de": "Tafkheem – alle Buchstaben",
      "en": "Tafkheem – all letters",
      "tasks": [
        45,
        53,
        61,
        9,
        12,
        11,
        "..."
      ]
    }
  ]
}

```

id	replaces	exercises	what it is
tafkheem-1-advanced	lessons 1–7	64	all heavy letters in one lesson
qalqala-1-advanced	lessons 10–14	64	Qalqalah inside the word
qalqala-2-advanced	lessons 10–14	40	Qalqalah at the end of the verse

Whether to use them is a per-learner setting. When a group is active its lessons take the place of the ones in `replaces` and the numbering closes up: with both groups on, lesson 1 is the combined Tafkheem lesson, lesson 2 is what used to be 8, lessons 4 and 5 are the two Qalqalah lessons, lesson 6 is what used to be 15 — 31 lessons instead of 40.

```

function lessonList(advanced /* the entries you want active */) {
  const out = []; let old = 1, n = 1;
  while (old <= 40) {
    const here = advanced.filter(a => a.replaces[0] === old);
    if (here.length) {
      for (const a of here) out.push({ number: n++, combined: a });
      old = here[0].replaces.at(-1) + 1;
    } else {
      out.push({ number: n++, lesson: old });
      old++;
    }
  }
  return out;
}

```

Do not read the three `*-advanced.json` files alongside the seven lesson files. They are the same exercises a second time, written out for convenience. Merging all ten would double-count 168 exercises. Either use the seven plus `advanced.json`, or use the advanced files on their own.

The combined lessons are balanced deliberately: roughly equal numbers from each source lesson, the interaction types in the proportion of the pool, and the answers spread flatter than the pool — in the source, more than half the counting answers are zero, which would reward guessing. The exception is “Qalqalah at the end of the verse”: there are only 40 exercises in the whole corpus where the prolonged stop at the verse end actually produces a Qalqalah, so that lesson holds all 40 and no selection takes place.

9 · Rendering Qur’anic Arabic

The text is not ordinary Arabic prose. It is the Uthmani orthography of the muṣḥaf, and three things go wrong if you treat it as plain text.

The font

Use a muṣḥaf face. We ship `fonts/hafs-uthmanic.woff2` (KFGQPC HAFS Uthmanic Script, 61 kB subset) for the body of the text and `fonts/amiri-quran-arabic-400-normal.woff2` for one glyph only, the verse-number cartouche. A system Arabic font will render most of it, but the small superscript marks — the dagger alif ^ا, the small wāw and yā’ of the ṣila ^ة, the silent-letter circle [◌], the maddah ^{◌◌◌} — are frequently missing, and their absence changes what the learner sees.

The verse number

Indic digits at the end of a verse are its number, not part of the word. They belong inside the ornamented cartouche: put `U+06DD` immediately before the digits and a space before that. Split the tail off before you lay the text out:

```
const AYAH_TAIL = /[\u0660-\u0669\u06F0-\u06F9]+$/;

function splitAyah(text) {
  const m = text.match(AYAH_TAIL);
  if (!m) return [text, ''];
  return [text.slice(0, m.index).trimEnd(), '\u06DD' + m[0]]; // cartouche + digits
}
```

Render the second part in the cartouche font and the first part in the body font. Keep the digits as they are — do not convert them to Western numerals, and do not include them in a preview that shows only the first few words.

Direction

Set `direction: rtl` and `unicode-bidi: isolate` on every element that holds Arabic. Without the isolation, a Latin label or a Western numeral next to the text drags punctuation to the wrong end. Give the text generous line height — 1.9 to 2.2 — because the superscript marks sit high and collide otherwise.

Do not normalise, do not strip

Every mark in the text is deliberate. Removing diacritics to “simplify” matching, or running the text through a normaliser that recomposes sequences, will silently invalidate every `spots` offset and can change the reading. Compare strings byte for byte, or not at all.

10 · Invariants worth asserting

These all hold for the shipped corpus. Assert them once when you load the data; each one has caught a real bug at some point.

Invariant	Why it matters
<code>id</code> is unique across all seven files	Two exercises once collided at a file seam and shared a recording.
<code>answer</code> is never empty, except for <code>mark_verse</code>	An empty answer elsewhere means a missing key, not “none”.
<code>yes_no</code> , <code>select_count</code> , <code>select_position</code> have exactly one answer id	Anything else is a data error.
<code>select</code> with <code>multiple: false</code> has exactly one answer id	Twelve exercises once had two.
every answer id exists in <code>options</code> (or <code>items</code>)	Guards against reordering damage.
<code>mark_verse</code> answers are exact substrings of <code>subject.text</code>	They are matched literally, not fuzzily.
<code>matching</code> pairs cover every item exactly once	Otherwise a bucket stays unexplained.
<code>subject</code> is null only for <code>matching</code>	Everything else has something to show.
<code>options</code> is empty only for <code>mark_verse</code>	
<code>multiple</code> is present exactly on <code>select</code>	
<code>text.slice(start, end) === spot.text</code> for every span	Catches encoding damage immediately.
audio filenames match <code><id>(_n _prompt)?.wav</code>	
no fixed token carries an audio filename	37 once did; there is nothing to record.
no Latin letters in any text outside the fixed tokens	Keeps the corpus language-neutral.

11 · Appendix: inventory

Exercises per file and interaction type

File	Lessons	n	yes_no	count	pos	select	match	matching	mark
tafkheem	1–9	575	71	144	72	216	0	72	0
qalqala	10–14	319	41	80	40	118	0	40	0
idgham	15–21	288	108	108	0	0	72	0	0
ikhfa-idgham	22–26	294	67	91	0	23	67	2	44
iqḷab	27–28	128	24	34	0	16	22	2	30
madd	29–34	256	160	96	0	0	0	0	0
waqf	35–40	176	0	0	0	80	0	0	96
total	1–40	2036	471	553	112	453	161	116	170

Exercises per lesson

1 64 2 64 3 64 4 63 5 64 6 64 7 64 8 64

9 64 10 64 11 63 12 64 13 64 14 64 15 36 16 60

17 40 18 36 19 36 20 40 21 40 22 47 23 55 24 64

25 64 26 64 27 64 28 64 29 48 30 48 31 48 32 48

33 48 34 16 35 32 36 32 37 32 38 32 39 32 40 16

Lessons 15–23 and 29–40 are smaller than 64 by design: the material for those rules is finite. Lesson 34, the opening letters of the suras, has only 16 exercises because the Qur'an contains only fourteen distinct openings; lesson 40, the two-way pause, has 16 because the mark occurs at only a handful of places.

Where to look when something is unclear

- The exercise browser at [/browser/](#) shows every exercise with its answer key, its spans and the anomalies it can detect — useful for checking what a record is supposed to look like on screen.
- The app preview at [/](#) is a working implementation of everything in this document: question text from the catalogue, spans as highlights, the combined lessons, all seven interaction types.
- [/fragen/](#) is where the wordings are edited.